

CineMatch

A Hybrid Movie Recommendation System

Complete project pipeline, from an empty folder to a deployed web app

Item	Detail
Skill gap this fills	Recommendation Systems (the remaining item in your portfolio gap list)
Dataset	MovieLens "ml-latest-small" (about 100k ratings, 610 users, 9.7k movies)
Models built	Popularity, bias baseline, content-based, item-item and user-user CF, matrix factorization (from scratch), hybrid
Final product	Flask web app + evaluation report + Power BI dashboard + GitHub repo
Total effort	About 3 weeks at 2 hours per day (about 40 hours)
Difficulty	Intermediate. Needs only pandas, numpy, and basic Flask, which you already know

How to use this document: go phase by phase, in order. Each phase has a goal, the exact steps, working code, the files you must produce, a checkpoint to confirm you are done, and the mistakes people usually make. Do not skip the evaluation phases; they are what makes this a data-science project rather than a tutorial copy.

Contents

1. Project Overview	3
Phase 1: Environment, Repository and Configuration	5
Phase 2: Data Acquisition and Exploratory Data Analysis	7
Phase 3: Cleaning, Splitting and Matrix Construction	9
Phase 4: Evaluation Framework and Baselines	12
Phase 5: Content-Based Filtering	17
Phase 6: Neighborhood Collaborative Filtering (Item-kNN and User-kNN)	19
Phase 7: Matrix Factorization from Scratch	22
Phase 8: Hybrid Model and Cold-Start Handling	25
Phase 9: Final Experiments, Error Analysis and the Test Evaluation	28
Phase 10: Flask Web Application	30
Phase 11: Power BI Dashboard	33
Phase 12: Documentation, Testing, Deployment and Resume	35
Phase 13: Optional Extensions (Pick One or Two)	37
Appendix A: Formula and Concept Cheat Sheet	38
Appendix B: Common Pitfalls and How to Avoid Them	38
Appendix C: Interview Questions with Short Answers	40
Appendix D: Master Checklist	41

1. Project Overview

1.1 What a recommendation system is

A recommendation system predicts which items a user will like, using the history of what many users have rated, watched, or bought. It powers Netflix rows, YouTube suggestions, Amazon "customers also bought", and Spotify playlists. There are two different questions it can answer, and you must keep them separate:

- **Rating prediction:** "What rating would user U give movie M?" Judged by RMSE and MAE.
- **Top-N recommendation:** "Which 10 movies should I show user U?" Judged by Precision@K, Recall@K, NDCG@K. This is what real products care about, and it is a different problem from rating prediction.

1.2 Why this project suits your level

- You already know pandas, scikit-learn, Flask, and Power BI. This project reuses all four and adds only the recommender-specific ideas.
- The dataset is small enough to train every model on a laptop in seconds to minutes. No GPU needed.
- It is large enough to show the real difficulties: sparsity, long-tail popularity, cold start, and evaluation leakage.
- You implement the core algorithms yourself (item-kNN, matrix factorization), so you can explain them in an interview instead of just calling a library.

1.3 Final deliverables

1. A GitHub repository with clean code in `src/`, notebooks in `notebooks/`, tests, and a README.
2. A model comparison table (RMSE, MAE, Precision@10, Recall@10, NDCG@10, coverage, novelty) for every model.
3. A hybrid recommender that beats every single model on NDCG@10 (this is the project headline).
4. A Flask web app with: movie search, "similar movies", recommendations for an existing user, and an onboarding flow for a brand-new user (cold start).
5. A Power BI dashboard of the data exploration and model results.
6. A deployed live demo link and a 1-page project summary for your resume.

1.4 Tech stack

Layer	Tools	Why
Language	Python 3.10 or newer	Standard for data science
Data	pandas, numpy, scipy.sparse	Rating matrices are 98 percent empty, so sparse storage is essential
ML helpers	scikit-learn (TF-IDF, cosine), numba	numba makes the from-scratch SGD loop fast
Plots	matplotlib, seaborn	EDA and error analysis
Serving	Flask, gunicorn, joblib	You already know Flask; joblib saves trained models
Frontend	HTML, Bootstrap, vanilla JS (fetch)	No heavy framework needed
BI	Power BI Desktop	Dashboard on exported CSVs
Quality	pytest, git, GitHub	Tests for metrics, version control

1.5 Pipeline at a glance

Phase	What you do	Output
1	Environment, repo structure, config	Working repo skeleton
2	Download data, exploratory analysis	EDA notebook, 8 plots, written findings
3	Clean data, split train/val/test, build sparse matrices	Processed data + index maps
4	Evaluation framework and baselines	metrics.py, 3 baseline results
5	Content-based filtering (TF-IDF on genres and tags)	Content model + results
6	Neighborhood collaborative filtering	Item-kNN, User-kNN + results
7	Matrix factorization from scratch	Biased MF model + training curves
8	Hybrid model and cold-start handling	Hybrid + new-user fold-in
9	Experiments, tuning, error analysis	Final results table + analysis
10	Flask web application	Running local app
11	Power BI dashboard	.pbix file + screenshots
12	Documentation, tests, deployment, resume	Live demo + polished repo
13	Optional extensions (including a cricket version)	Bonus portfolio pieces

1.6 Suggested schedule

Week	Phases	Target by end of week
Week 1	1, 2, 3, 4, 5	Clean data, evaluation harness, baselines, content-based model all producing numbers
Week 2	6, 7, 8, 9	All models trained, hybrid beating individuals, error analysis done
Week 3	10, 11, 12	Web app live, dashboard built, README and resume bullets done

Golden rule for the whole project: never look at the test set while making decisions. All tuning (hyperparameters, hybrid weights, model choice) uses the validation set. The test set is used exactly once at the very end of Phase 9 to produce the final numbers you report.

Phase 1: Environment, Repository and Configuration

Estimated time: 2 to 3 hours

Goal of this phase: Have a clean, reproducible project skeleton so that later phases only add code, never reorganise.

1.1 Create the folder structure

```
cinematch/
data/
  raw/                # downloaded zip contents (never edited, git-ignored)
  processed/          # cleaned splits, sparse matrices
artifacts/            # trained models, similarity tables, id maps (used by the web app)
notebooks/
  01_edu.ipynb
  02_preprocessing.ipynb
  03_baselines.ipynb
  04_content_based.ipynb
  05_knn_cf.ipynb
  06_matrix_factorization.ipynb
  07_hybrid.ipynb
  08_final_evaluation.ipynb
src/
  __init__.py
  config.py           # paths + constants
  data.py             # loading, cleaning, splitting
  metrics.py          # RMSE, MAE, ranking metrics, coverage, novelty
  evaluate.py         # evaluate any model on val/test
  models/
    __init__.py
    base.py           # Recommender interface
    baselines.py      # Random, Popularity, Bias
    content.py        # Content-based
    knn.py            # Item-kNN, User-kNN
    mf.py             # Biased MF (numba SGD)
    hybrid.py         # weighted blend + cold start
  app/
    app.py            # Flask app
    templates/index.html
    static/js/main.js
    static/css/style.css
  reports/            # results CSVs, plots, final write-up
tests/
  test_metrics.py
requirements.txt
README.md
.gitignore
```

1.2 Create the virtual environment and install packages

A virtual environment keeps this project's libraries separate from everything else on your machine. Run these commands in the project root.

```
python -m venv venv

# Windows (PowerShell / cmd)
venv\Scripts\activate
# Mac / Linux
source venv/bin/activate

pip install pandas numpy scipy scikit-learn matplotlib seaborn
pip install flask gunicorn joblib jupyter tqdm pytest numba pyarrow
pip freeze > requirements.txt

git init
```

1.3 Write .gitignore

```
venv/  
__pycache__/  
.ipynb_checkpoints/  
data/raw/  
*.pbix.bak  
.DS_Store
```

Keep artifacts/ out of .gitignore only if the files are small (under about 50 MB in total). You will need them for deployment in Phase 12.

1.4 Write config.py

Every constant lives in one file. This prevents the classic bug where the notebook uses a threshold of 4.0 and the web app uses 3.5.

```
# src/config.py  
from pathlib import Path  
  
ROOT = Path(__file__).resolve().parents[1]  
DATA_RAW = ROOT / "data" / "raw" / "ml-latest-small"  
DATA_PROC = ROOT / "data" / "processed"  
ARTIFACTS = ROOT / "artifacts"  
REPORTS = ROOT / "reports"  
  
SEED = 42  
REL_THRESHOLD = 4.0      # a rating >= 4.0 counts as "relevant" for ranking metrics  
TEST_FRAC = 0.20        # last 20% of each user's ratings (by time) -> test  
VAL_FRAC = 0.10         # last 10% of the remaining -> validation  
TOP_K = 10  
RATING_MIN, RATING_MAX = 0.5, 5.0  
  
for d in (DATA_PROC, ARTIFACTS, REPORTS):  
    d.mkdir(parents=True, exist_ok=True)
```

1.5 Checkpoint

- `python -c "from src.config import ROOT; print(ROOT)"` prints your project path.
- The first git commit exists with the skeleton and requirements.txt.
- Jupyter opens and can import `src` (start notebooks with `import sys; sys.path.append("../")`).

1.6 Common mistakes

- Putting all code in one giant notebook. You will not be able to reuse it in the Flask app. Put logic in `src/`, use notebooks only to call it and show plots.
- Hard-coding absolute paths like `C:\Users\...` in code. Always build paths from `config.py`.

Phase 2: Data Acquisition and Exploratory Data Analysis

Estimated time: 4 to 5 hours

Goal of this phase: Understand the data deeply enough to explain why each modelling decision later is justified.

2.1 Download the dataset

Download from the GroupLens site (grouplens.org/datasets/movielens/latest/), file `ml-latest-small.zip`. Unzip it into `data/raw/` so that the path is `data/raw/ml-latest-small/ratings.csv`.

File	Columns	Meaning
<code>ratings.csv</code>	<code>userId</code> , <code>movieId</code> , <code>rating</code> , <code>timestamp</code>	About 100k explicit ratings from 0.5 to 5.0 in half-star steps; timestamp is Unix seconds
<code>movies.csv</code>	<code>movieId</code> , <code>title</code> , <code>genres</code>	Title includes release year in brackets; genres separated by the pipe symbol
<code>tags.csv</code>	<code>userId</code> , <code>movieId</code> , <code>tag</code> , <code>timestamp</code>	Free-text tags users attached to movies (a few thousand rows)
<code>links.csv</code>	<code>movieId</code> , <code>imdbId</code> , <code>tmdbId</code>	IDs for fetching posters or metadata from IMDb/TMDB (used optionally in the web app)

Every user in this dataset has at least 20 ratings. That is a convenience (no users with zero history) but also means the data hides the true cold-start problem. You will simulate cold start yourself in Phase 8.

2.2 Loader code

```
# src/data.py (first part)
import re
import numpy as np
import pandas as pd
from .config import DATA_RAW

def load_raw():
    ratings = pd.read_csv(DATA_RAW / "ratings.csv")
    movies = pd.read_csv(DATA_RAW / "movies.csv")
    tags = pd.read_csv(DATA_RAW / "tags.csv")
    links = pd.read_csv(DATA_RAW / "links.csv")
    return ratings, movies, tags, links
```

2.3 The questions your EDA notebook must answer

Write each question as a markdown heading in the notebook, then code, then one or two sentences of finding. These sentences are what you will later put in the README.

1. **Basic shape:** how many ratings, users, movies? Any missing values? Any duplicate (user, movie) pairs?
2. **Rating distribution:** plot a bar chart of counts per rating value. Finding to look for: people rate whole stars more often than half stars, and ratings skew positive (mean around 3.5).
3. **Sparsity:** compute $1 - (\text{ratings} / (\text{users} \times \text{movies}))$. It is about 98 percent. Explain what this means for algorithms: most user-item pairs have no data, so naive methods fail.
4. **Ratings per user:** histogram on a log scale. A few power users rate thousands of movies while most rate 20 to 100.
5. **Ratings per movie (long tail):** sort movies by count and plot the curve. A small number of blockbusters get most ratings; thousands of movies have 1 to 3 ratings. This is the single most important fact for recommender design.

6. **Popularity vs quality:** scatter of rating count vs mean rating. Movies with very few ratings have extreme means (all 5.0 or all 0.5), which is noise, not quality. This motivates the Bayesian average in Phase 4.
7. **Genres:** count of movies per genre and mean rating per genre.
8. **User behaviour:** distribution of per-user mean rating. Some users are generous (mean 4.2) and some harsh (mean 2.8). This motivates user bias terms.
9. **Time:** ratings per year, and the span of each user's activity. This motivates the time-based split in Phase 3.
10. **Release-year effect:** mean rating by movie release year (extract year from the title).

2.4 Key EDA code

```
ratings, movies, tags, links = load_raw()

print(ratings.shape, movies.shape, tags.shape)
print(ratings.isna().sum())
print("duplicates:", ratings.duplicated(["userId", "movieId"]).sum())

n_users = ratings.userId.nunique()
n_items = ratings.movieId.nunique()
sparsity = 1 - len(ratings) / (n_users * n_items)
print(f"users={n_users} items={n_items} sparsity={sparsity:.4f}")

# rating distribution
ratings.rating.value_counts().sort_index().plot(kind="bar", title="Rating distribution")

# long tail
counts = ratings.groupby("movieId").size().sort_values(ascending=False).values
plt.plot(counts); plt.yscale("log"); plt.title("Ratings per movie (sorted)")

# share of ratings from the top 10% of movies
top10 = counts[: int(0.1 * len(counts))].sum() / counts.sum()
print(f"top 10% of movies hold {top10:.1%} of ratings")

# popularity vs mean rating
stats = ratings.groupby("movieId").rating.agg(["count", "mean"])
stats.plot.scatter(x="count", y="mean", alpha=0.3, logx=True)

# genres (one row per movie-genre)
g = movies.assign(genre=movies.genres.str.split("|")).explode("genre")
g = g[g.genre != "(no genres listed)"]
print(g.genre.value_counts())
genre_mean = (ratings.merge(g[["movieId", "genre"]], on="movieId")
               .groupby("genre").rating.agg(["mean", "count"]).sort_values("mean"))
```

2.5 Checkpoint

- Notebook 01_eda.ipynb with at least 8 clean, labelled plots saved to reports/eda/.
- A short markdown list "Key findings" with at least 6 bullet points that include real numbers from your run.
- You can answer out loud: "Why can't I just use the average rating to recommend movies?" (answer: low-count noise and zero personalisation).

2.6 Common mistakes

- Plotting without log scale on long-tail data, which makes every plot look like a single spike.
- Skipping the time analysis and later using a random split, which leaks future information into training.

Phase 3: Cleaning, Splitting and Matrix Construction

Estimated time: 4 to 5 hours

Goal of this phase: Produce train, validation and test sets without leakage, plus the sparse matrices and ID maps every model needs.

3.1 Cleaning steps

1. Drop duplicate (userId, movieId) rows, keeping the latest rating.
2. Extract the release year from the title with a regex, then keep the clean title separately.
3. Replace the genre string "(no genres listed)" with an empty list.
4. Aggregate tags per movie into one lowercase string (used in Phase 5).
5. Do **not** remove low-activity users or rare movies globally. Filtering the data you will be evaluated on makes results look better than they are. If a model needs filtering internally (for example, ignoring items with fewer than 5 ratings when computing similarity), do it inside that model.

```
def clean_movies(movies):
    m = movies.copy()
    m["year"] = m.title.str.extract(r"((\d{4}))\s*$").astype("float")
    m["clean_title"] = m.title.str.replace(r"\s*(\d{4})\s*$", "", regex=True)
    m["genre_list"] = m.genres.apply(
        lambda s: [] if s == "(no genres listed)" else s.split("|")
    )
    return m

def clean_ratings(r):
    return (r.sort_values("timestamp")
            .drop_duplicates(["userId", "movieId"], keep="last")
            .reset_index(drop=True))

def aggregate_tags(tags):
    t = tags.dropna(subset=["tag"]).copy()
    t["tag"] = t.tag.str.lower().str.strip()
    return t.groupby("movieId").tag.apply(lambda s: " ".join(s)).rename("tag_text")
```

3.2 Why a per-user temporal split

A random split is the most common beginner error. It puts a user's 2018 ratings in training and their 2016 ratings in test, so the model effectively sees the future. In a real product you only know the past and must predict what happens next. The correct protocol:

1. For every user, sort their ratings by timestamp.
2. The most recent 20 percent of that user's ratings go to **test**.
3. From the remaining earlier ratings, the most recent 10 percent go to **validation**; the rest is **train**.
4. Every user appears in all three sets (possible because everyone has 20 or more ratings).

```
import numpy as np
from .config import TEST_FRAC, VAL_FRAC

def temporal_split_per_user(df, frac):
    """Return (earlier, later); `later` is the last `frac` of each user's ratings."""
    df = df.sort_values(["userId", "timestamp"]).copy()
    n = df.groupby("userId")["userId"].transform("size")
    pos_from_end = df.groupby("userId").cumcount(ascending=False)
    n_later = np.ceil(n * frac).astype(int)
    is_later = pos_from_end < n_later
    return df[~is_later].copy(), df[is_later].copy()

def make_splits(ratings):
    train_val, test = temporal_split_per_user(ratings, TEST_FRAC)
```

```
train, val = temporal_split_per_user(train_val, VAL_FRAC)
return train, val, test
```

Ratings with identical timestamps (a user rating many movies in the same second) are ordered arbitrarily. That is fine for this project; mention it in the README as a known limitation.

3.3 ID mapping

Raw movieids are not contiguous (they go up to about 193,000 but only 9,700 exist). Matrices need row/column numbers 0..n-1. Build the mapping from the **full** set of users and movies (this is just bookkeeping, not leakage) so that val/test items that never appear in train still have an index.

```
def build_index_maps(ratings, movies):
    user_ids = np.sort(ratings.userId.unique())
    item_ids = np.sort(movies.movieId.unique())
    user2idx = {u: i for i, u in enumerate(user_ids)}
    item2idx = {m: i for i, m in enumerate(item_ids)}
    return user_ids, item_ids, user2idx, item2idx

def add_indices(df, user2idx, item2idx):
    df = df.copy()
    df["u"] = df.userId.map(user2idx).astype(np.int32)
    df["i"] = df.movieId.map(item2idx).astype(np.int32)
    return df
```

3.4 Sparse user-item matrix

```
import scipy.sparse as sp

def to_csr(df, n_users, n_items, value_col="rating"):
    return sp.csr_matrix(
        (df[value_col].values.astype(np.float32), (df.u.values, df.i.values)),
        shape=(n_users, n_items))
```

A CSR matrix stores only the non-zero entries, so the 610 x 9,742 matrix uses a few hundred KB instead of 24 MB, and the saving becomes decisive on larger datasets. Row slicing (all items a user rated) is fast in CSR. For column slicing (all users who rated an item) convert once with `.tocsc()`.

3.5 Save everything

```
# notebooks/02_preprocessing.ipynb
ratings, movies, tags, links = load_raw()
ratings = clean_ratings(ratings)
movies = clean_movies(movies)
movies = movies.merge(aggregate_tags(tags), on="movieId", how="left")
movies["tag_text"] = movies.tag_text.fillna("")

user_ids, item_ids, user2idx, item2idx = build_index_maps(ratings, movies)
ratings = add_indices(ratings, user2idx, item2idx)
movies["i"] = movies.movieId.map(item2idx)
movies = movies.sort_values("i").reset_index(drop=True)  # row k == item index k

train, val, test = make_splits(ratings)
print(len(train), len(val), len(test))
for name, d in [("train", train), ("val", val), ("test", test)]:
    d.to_parquet(DATA_PROC / f"{name}.parquet")
movies.to_parquet(DATA_PROC / "movies.parquet")

import joblib
joblib.dump(dict(user_ids=user_ids, item_ids=item_ids,
                 user2idx=user2idx, item2idx=item2idx), ARTIFACTS / "id_maps.joblib")
```

3.6 Sanity tests to run before moving on

- Max timestamp in train is not greater than min timestamp in test for any given user (loop over users or use `groupby`).

- No (u, i) pair appears in two splits.
- train + val + test row count equals the cleaned ratings row count.
- Every row of movies has i equal to its row position.
- Report how many test ratings involve an item with zero train ratings (these "cold items" cannot be scored by collaborative models).

3.7 Checkpoint and mistakes

- **Done when:** three parquet files, movies.parquet, id_maps.joblib exist and the sanity tests pass.
- **Mistake:** fitting the ID map on train only; then val items get NaN indices and crash later.
- **Mistake:** using the test set to compute anything (item means, popularity). Item statistics must come from train only.

Phase 4: Evaluation Framework and Baselines

Estimated time: 5 to 6 hours

Goal of this phase: Build one reusable evaluation harness, then measure three simple baselines. Every later model must beat them.

4.1 Why baselines come first

A recommender with RMSE 0.92 sounds fine until you learn that "predict each user's average" already gets 0.93. Baselines tell you how much of your result comes from real modelling and how much from easy structure in the data. In ranking, a plain "most popular movies" list is notoriously hard to beat, so it is your most important baseline.

4.2 The common model interface

Define one interface so the evaluator, the hybrid model, and the web app can treat every model identically.

```
# src/models/base.py
import numpy as np

class Recommender:
    name = "base"

    def fit(self, train_df, n_users, n_items):
        raise NotImplementedError

    def predict(self, u, i):
        """u, i: int arrays of equal length -> predicted ratings (float array)."""
        raise NotImplementedError

    def score_user(self, u):
        """Return a float array of length n_items: higher = more recommended.
        Seen-item masking is done by the evaluator, not the model."""
        raise NotImplementedError
```

4.3 Metrics explained (read carefully)

Metric	Question it answers	Formula in words
RMSE	How far off are predicted ratings, penalising big errors?	Square root of the mean of (true minus predicted) squared
MAE	Average absolute error	Mean of true minus predicted
Precision@K	Of the K items shown, what fraction did the user actually like?	Hits in top K divided by K
Recall@K	Of all items the user liked in test, how many did we surface in top K?	Hits in top K divided by number of relevant test items
NDCG@K	Like precision, but hits near the top count more	DCG divided by ideal DCG, where DCG = sum of hit / log2(rank+1)
MAP@K	Average precision across hit positions	Mean over users of average precision at each hit
Coverage	What fraction of the catalogue ever gets recommended?	Distinct recommended items divided by total items
Novelty	Do we recommend obscure items or only blockbusters?	Mean of -log2(popularity share) over recommended items

Definition of "relevant": a test item is relevant to a user if their test rating is at least 4.0 (REL_THRESHOLD in config). Candidates for ranking are all items the user has **not** rated in train (or validation, when evaluating on test). Absolute ranking scores will look small (Precision@10 of 0.05 to 0.15 is normal) because each user has only a handful of relevant test items among thousands of candidates.

4.4 metrics.py

```
# src/metrics.py
import numpy as np

def rmse(y, p):
    y, p = np.asarray(y, float), np.asarray(p, float)
    return float(np.sqrt(np.mean((y - p) ** 2)))

def mae(y, p):
    y, p = np.asarray(y, float), np.asarray(p, float)
    return float(np.mean(np.abs(y - p)))

def ranking_metrics(recs, relevant, k):
    """recs: ranked list of item indices (len >= k). relevant: set of item indices.
    Returns precision, recall, ndcg, average_precision for one user."""
    recs = list(recs[:k])
    hits = np.array([1.0 if r in relevant else 0.0 for r in recs])
    n_rel = len(relevant)
    precision = hits.sum() / k
    recall = hits.sum() / n_rel if n_rel else 0.0
    discounts = 1.0 / np.log2(np.arange(2, len(recs) + 2))
    dcg = float((hits * discounts).sum())
    ideal_hits = min(n_rel, k)
    idcg = float((1.0 / np.log2(np.arange(2, ideal_hits + 2))).sum())
    ndcg = dcg / idcg if idcg > 0 else 0.0
    # average precision
    cum = np.cumsum(hits)
    ap = float(((cum / np.arange(1, len(recs) + 1)) * hits).sum() / min(n_rel, k)) if n_rel else 0.0
    return precision, recall, ndcg, ap

def coverage(all_recs, n_items):
    return len(set(i for recs in all_recs for i in recs)) / n_items

def novelty(all_recs, item_pop_share):
    """item_pop_share: array, fraction of train ratings that belong to each item."""
    vals = [-np.log2(item_pop_share[i] + 1e-12) for recs in all_recs for i in recs]
    return float(np.mean(vals))
```

4.5 evaluate.py - the harness

```
# src/evaluate.py
import numpy as np
from .metrics import rmse, mae, ranking_metrics, coverage, novelty
from .config import REL_THRESHOLD, TOP_K

def evaluate_rating(model, eval_df, rmin=0.5, rmax=5.0):
    pred = model.predict(eval_df.u.values, eval_df.i.values)
    pred = np.clip(pred, rmin, rmax)
    return {"rmse": rmse(eval_df.rating, pred), "mae": mae(eval_df.rating, pred)}

def evaluate_ranking(model, seen_df, eval_df, n_items, k=TOP_K, thr=REL_THRESHOLD):
    """seen_df: ratings that must be excluded from recommendations
    (train when scoring val; train+val when scoring test)."""
    seen = seen_df.groupby("u").i.apply(np.array).to_dict()
    relevant = (eval_df[eval_df.rating >= thr].groupby("u").i.apply(set).to_dict())
    pop = np.bincount(seen_df.i.values, minlength=n_items).astype(float)
    pop_share = pop / pop.sum()
    P, R, N, A, all_recs = [], [], [], [], []
    for u, rel in relevant.items():
        s = model.score_user(u).astype(float).copy()
        if u in seen:
            s[seen[u]] = -np.inf
        top = np.argpartition(-s, k)[:k]
        top = top[np.argsort(-s[top])]
        p, r, n, a = ranking_metrics(top, rel, k)
        P.append(p); R.append(r); N.append(n); A.append(a)
        all_recs.append(top.tolist())
    return {"precision@{k}": np.mean(P), f"recall@{k}": np.mean(R),
            f"ndcg@{k}": np.mean(N), f"map@{k}": np.mean(A),
            "coverage": coverage(all_recs, n_items),
            "novelty": novelty(all_recs, pop_share),
            "users_evaluated": len(relevant)}
```

Note that users with no relevant test items are skipped (you cannot compute recall for them). Report `users_evaluated` so the reader knows the denominator.

4.6 Unit tests for metrics (do this now, not later)

```
# tests/test_metrics.py
import numpy as np
from src.metrics import rmse, ranking_metrics

def test_rmse_zero():
    assert rmse([1, 2, 3], [1, 2, 3]) == 0

def test_perfect_ranking():
    p, r, n, a = ranking_metrics([5, 7, 9], {5, 7, 9}, 3)
    assert p == 1 and r == 1 and abs(n - 1) < 1e-9 and abs(a - 1) < 1e-9

def test_no_hits():
    p, r, n, a = ranking_metrics([1, 2, 3], {9}, 3)
    assert p == 0 and r == 0 and n == 0

def test_ndcg_prefers_early_hit():
    _, _, n_early, _ = ranking_metrics([1, 2, 3], {1}, 3)
    _, _, n_late, _ = ranking_metrics([2, 3, 1], {1}, 3)
    assert n_early > n_late
```

4.7 Three baselines

Baseline A: Random

Scores are random numbers. This gives the floor: any model must beat it by a wide margin. It also verifies your evaluator is sane (random NDCG@10 should be tiny, close to 0.001 to 0.005).

Baseline B: Popularity (Bayesian average)

Raw mean rating is noisy for movies with few ratings. The Bayesian (damped) average pulls low-count movies toward the global mean. For item i with n_i ratings and sum S_i : $\text{score} = (S_i + m \times \mu) / (n_i + m)$, where μ is the global mean and m is the damping strength (try 10 to 25). Also try pure rating count as an alternative popularity score; for top-N ranking, count often wins, because users mostly watch popular movies. Report both.

Baseline C: Bias model (global mean + user bias + item bias)

Prediction = $\mu + b_u + b_i$. It captures "this user rates generously" and "this movie is generally loved". It is a surprisingly strong rating predictor and is also the base layer under the better models. Estimate the biases with regularisation (Koren, 2010): $b_i = \text{sum over users of } (r_{ui} - \mu) / (\lambda_1 + n_i)$, then $b_u = \text{sum over items of } (r_{ui} - \mu - b_i) / (\lambda_2 + n_u)$. Typical values are $\lambda_1 = 10$, $\lambda_2 = 15$.

```
# src/models/baselines.py
import numpy as np
import pandas as pd
from .base import Recommender

class RandomRec(Recommender):
    name = "random"
    def fit(self, train, n_users, n_items):
        self.n_items = n_items; self.rng = np.random.default_rng(0)
        self.mu = train.rating.mean(); return self
    def predict(self, u, i): return np.full(len(u), self.mu)
    def score_user(self, u): return self.rng.random(self.n_items)

class Popularity(Recommender):
    def __init__(self, mode="bayes", m=15):
        self.mode, self.m = mode, m; self.name = f"popularity_{mode}"
    def fit(self, train, n_users, n_items):
        cnt = np.bincount(train.i, minlength=n_items).astype(float)
        sm = np.bincount(train.i, weights=train.rating, minlength=n_items)
        self.mu = train.rating.mean()
        self.bayes = (sm + self.m * self.mu) / (cnt + self.m)
        self.count = cnt
        return self
    def predict(self, u, i): return self.bayes[i]
    def score_user(self, u):
        return self.count if self.mode == "count" else self.bayes

class BiasModel(Recommender):
    name = "bias"
    def __init__(self, lam_i=10, lam_u=15): self.lam_i, self.lam_u = lam_i, lam_u
    def fit(self, train, n_users, n_items):
        self.mu = train.rating.mean()
        d = train.assign(r0=train.rating - self.mu)
        gi = d.groupby("i").r0.agg(["sum", "count"])
        self.bi = np.zeros(n_items)
        self.bi[gi.index] = gi["sum"] / (self.lam_i + gi["count"])
        d["r1"] = d.r0 - self.bi[d.i.values]
        gu = d.groupby("u").r1.agg(["sum", "count"])
        self.bu = np.zeros(n_users)
        self.bu[gu.index] = gu["sum"] / (self.lam_u + gu["count"])
        return self
    def predict(self, u, i): return self.mu + self.bu[u] + self.bi[i]
    def score_user(self, u): return self.bi # user bias does not change the ranking
```

4.8 Run and record results

```
import pandas as pd
from src.evaluate import evaluate_rating, evaluate_ranking

results = []
def log(model, split, metrics, **params):
    for k, v in metrics.items():
        results.append(dict(model=model.name, split=split, metric=k,
                           value=float(v), params=str(params)))

for model in [RandomRec(), Popularity("bayes"), Popularity("count"), BiasModel()]:
    model.fit(train, n_users, n_items)
    log(model, "val", evaluate_rating(model, val))
    log(model, "val", evaluate_ranking(model, train, val, n_items))

pd.DataFrame(results).to_csv(REPORTS / "results_val.csv", index=False)
```

Use the exact column schema **model, split, metric, value, params**. In Phase 11 Power BI will read this file directly, so keeping it tidy now saves rework.

4.9 What numbers to expect (ballpark only)

Model	RMSE	NDCG@10	Comment
Random	about 1.4 or higher if you predict random ratings	near 0.00x	Floor
Popularity (count)	n/a	noticeably above random	Strong baseline for ranking
Bias model	roughly 0.88 to 0.95	poor (same list for everyone)	Strong baseline for rating

Your exact numbers will differ with the split and seed. What matters is the ordering. If random beats popularity in ranking, you have a bug (usually seen items not being masked, or user/item indices mixed up).

4.10 Checkpoint and mistakes

- **Done when:** pytest passes, results_val.csv has rows for four baselines, and you can explain each metric without notes.
- **Mistake:** forgetting to mask already-seen items, so the model "recommends" movies the user already rated. Metrics become meaningless.
- **Mistake:** clipping predictions before computing ranking scores (not needed) or forgetting to clip before RMSE (needed).
- **Mistake:** evaluating on val with train+val as the seen set. For val use train only; for final test use train+val.

Phase 5: Content-Based Filtering

Estimated time: 4 to 5 hours

Goal of this phase: Recommend movies similar to what a user already liked, using only movie attributes (genres, tags, year). This model also solves new-item cold start.

5.1 The idea

Represent each movie as a vector of features. Represent each user as a weighted average of the vectors of movies they liked. Recommend movies whose vectors are closest to the user vector. No other users are involved, so it works for a brand-new movie the moment its genres are known.

5.2 Building movie vectors

1. **Genres:** multi-hot encode with TF-IDF so that rare genres (Film-Noir, IMAX) carry more weight than common ones (Drama).
2. **Tags:** TF-IDF on the aggregated tag text (limit vocabulary to the top 2,000 terms, drop words that appear in fewer than 2 movies).
3. **Decade:** a one-hot token such as "decade_1990". Optional, but it helps separate classics from modern films.
4. Weight the blocks (for example genres 1.0, tags 0.5, decade 0.3), stack them side by side, and L2-normalise every row so that a dot product equals cosine similarity.

```
# src/models/content.py
import numpy as np
import scipy.sparse as sp
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize
from .base import Recommender

def build_item_features(movies, w_genre=1.0, w_tag=0.5, w_decade=0.3):
    genre_doc = movies.genre_list.apply(lambda g: "|".join(g))
    gv = TfidfVectorizer(tokenizer=lambda s: [t for t in s.split("|") if t],
                        lowercase=False, token_pattern=None)
    G = gv.fit_transform(genre_doc)

    tv = TfidfVectorizer(stop_words="english", min_df=2, max_features=2000)
    T = tv.fit_transform(movies.tag_text)

    decade = movies.year.fillna(0).astype(int).apply(
        lambda y: f"decade_{(y // 10) * 10}" if y > 0 else "decade_unknown")
    dv = TfidfVectorizer(tokenizer=lambda s: [s], lowercase=False, token_pattern=None)
    D = dv.fit_transform(decade)

    X = sp.hstack([w_genre * G, w_tag * T, w_decade * D]).tocsr()
    return normalize(X), dict(genre=gv, tag=tv, decade=dv)

class ContentBased(Recommender):
    name = "content"
    def __init__(self, X): self.X = X # (n_items x d), rows L2-normalised
    def fit(self, train, n_users, n_items):
        self.n_users = n_users
        self.R = sp.csr_matrix((train.rating.values, (train.u.values, train.i.values)),
                               shape=(n_users, n_items))
        cnt = np.asarray((self.R != 0).sum(1)).ravel()
        sm = np.asarray(self.R.sum(1)).ravel()
        fallback = np.full(n_users, train.rating.mean())
        self.user_mean = np.divide(sm, cnt, out=fallback, where=cnt > 0)
        return self
    def profile(self, u):
        row = self.R[u]
        idx, r = row.indices, row.data
        w = r - self.user_mean[u] # liked above average -> positive weight
```

```

        if len(idx) == 0 or np.allclose(w, 0): w = np.ones(len(idx))
        return np.asarray(self.X[idx].T @ w).ravel()
    def score_user(self, u):
        return np.asarray(self.X @ self.profile(u)).ravel()
    def predict(self, u, i):
        return self.user_mean[u] # not a rating model; return neutral values
    def similar_items(self, i, n=10):
        s = np.asarray(self.X @ self.X[i].T.toarray()).ravel()
        s[i] = -1
        top = np.argsort(-s, n)[:n]
        return top[np.argsort(-s[top])], s[top[np.argsort(-s[top])]]

```

5.3 Design choices to explain in your README

- **Why subtract the user mean?** Without it, a movie rated 2 stars still pushes the profile toward its genres. Centering makes disliked movies push the profile away.
- **Why L2-normalise?** So long tag lists do not dominate and so cosine similarity is a single sparse matrix product.
- **Why no RMSE?** This model ranks; it does not output calibrated ratings. Mark RMSE as "n/a" in the results table instead of a fake number.

5.4 Experiments to run on validation data

Experiment	Variations	What you learn
Feature blocks	genres only; genres + tags; genres + tags + decade	Which metadata actually helps
Block weights	w_tag in 0, 0.25, 0.5, 1.0	Sensitivity
Profile weighting	mean-centered vs all-ones vs only items rated 4+	Effect of using negative feedback
Popularity prior	add small multiple of item popularity to score	Content alone tends to recommend obscure items

5.5 Qualitative check

Print 10 nearest neighbours of three movies you know (for example a Pixar film, a Nolan film, a classic). If "Toy Story" returns other animation/children/comedy titles, the vectors work. Paste these into the notebook; they are a great README exhibit.

5.6 Checkpoint and mistakes

- **Done when:** best content configuration chosen on validation; results appended to results_val.csv; neighbour lists look sensible.
- **Expected behaviour:** content-based NDCG is usually below popularity but above random, and its coverage and novelty are higher. That trade-off is a finding, so write it down.
- **Mistake:** forgetting that movies with no genres and no tags have an all-zero vector; guard against division by zero when normalising (sklearn handles it by leaving zeros).

Phase 6: Neighborhood Collaborative Filtering (Item-kNN and User-kNN)

Estimated time: 6 to 8 hours

Goal of this phase: Recommend using only rating behaviour: "people who liked what you liked also liked this". Implement it efficiently with sparse top-k similarity.

6.1 The idea

- **Item-item CF:** two movies are similar if the same users rated them similarly. To predict user u 's rating for movie i , look at the movies u has rated that are most similar to i and take a similarity-weighted average.
- **User-user CF:** two users are similar if they rated the same movies similarly. To predict u 's rating for i , look at the most similar users who rated i .
- Item-item is usually better and more stable when there are many more users than items; here users are fewer (610), so run both and compare. That comparison is a real finding.

6.2 Make ratings comparable: use residuals

A raw cosine similarity on ratings is dominated by the fact that everyone rates positively. Instead work with residuals from your Phase 4 bias model: $e_{ui} = r_{ui} - (\mu + b_u + b_i)$. A positive residual means "liked more than expected". Similarity is then computed between residual vectors, which isolates genuine taste.

```
def residual_matrix(train, bias):
    base = bias.mu + bias.bu[train.u.values] + bias.bi[train.i.values]
    e = (train.rating.values - base).astype(np.float32)
    E = sp.csr_matrix((e, (train.u.values, train.i.values)), shape=(bias.bu.size, bias.bi.size))
    B = sp.csr_matrix((np.ones(len(train), np.float32), (train.u.values, train.i.values)),
                      shape=E.shape)          # 1 where rated
    return E, B
```

6.3 Similarity with shrinkage and top-k pruning

Two movies co-rated by only 2 users can have cosine 1.0 by luck. Shrink similarities toward zero when the overlap is small: $\text{sim_shrunk} = \text{sim} \times n_{\text{common}} / (n_{\text{common}} + \lambda)$, with λ around 25 (tune: 10 to 100). Then keep only the top k neighbours per item ($k = 30$ to 100) so memory stays small.

```
# src/models/knn.py
import numpy as np
import scipy.sparse as sp
from sklearn.preprocessing import normalize
from .base import Recommender

def topk_similarity(M, B, k=50, shrink=25, chunk=1000):
    """M: (n x d) residual matrix (rows = entities to compare), B: binary rated matrix.
    Returns sparse (n x n) matrix keeping the top-k positive similarities per row."""
    Mn = normalize(M)
    n = M.shape[0]
    rows, cols, vals = [], [], []
    for s in range(0, n, chunk):
        e = min(s + chunk, n)
        S = (Mn[s:e] @ Mn.T).toarray()
        C = (B[s:e] @ B.T).toarray()
        S *= C / (C + shrink)
        S[np.arange(e - s), np.arange(s, e)] = 0
        S[S < 0] = 0
        kk = min(k, n - 1)
        top = np.argsort(-S, kk, axis=1)[: , :kk]
        v = np.take_along_axis(S, top, axis=1)
        keep = v > 0
        r = np.repeat(np.arange(s, e), kk).reshape(-1, kk)
        rows.append(r[keep]); cols.append(top[keep]); vals.append(v[keep])
```

```
return sp.csr_matrix((np.concatenate(vals), (np.concatenate(rows), np.concatenate(cols))),
                    shape=(n, n))
```

6.4 Item-kNN model

For item similarity the "entities" are items, so pass the transposed matrices (items x users).

```
class ItemKNN(Recommender):
    name = "item_knn"
    def __init__(self, bias, k=50, shrink=25, damp=1.0):
        self.bias, self.k, self.shrink, self.damp = bias, k, shrink, damp
    def fit(self, train, n_users, n_items):
        self.E, self.B = residual_matrix(train, self.bias)
        self.W = topk_similarity(self.E.T.tocsr(), self.B.T.tocsr(),
                                self.k, self.shrink) # items x items
        self.Wabs = abs(self.W)
        return self
    def _num_den(self, u):
        e_u = self.E[u].toarray().ravel() # residuals, 0 where unrated
        r_u = self.B[u].toarray().ravel()
        return self.W @ e_u, self.Wabs @ r_u
    def score_user(self, u):
        num, den = self._num_den(u)
        return num / (den + self.damp) # damped weighted average of neighbour residuals
    def predict(self, u, i):
        out = np.empty(len(u))
        cache = {}
        for n, (uu, ii) in enumerate(zip(u, i)):
            if uu not in cache: cache[uu] = self._num_den(uu)
            num, den = cache[uu]
            res = num[ii] / (den[ii] + self.damp)
            out[n] = self.bias.mu + self.bias.bu[uu] + self.bias.bi[ii] + res
        return out
```

Explanation of score_user: $\text{self.W} @ \text{e}_u$ gives, for every item i , the sum over neighbours j of $\text{sim}(i, j) \times \text{residual}(u, j)$. Dividing by the sum of absolute similarities of neighbours that u actually rated gives the weighted average. The damping constant keeps items with a single weak neighbour from jumping to the top.

6.5 User-kNN model

Same function, but entities are users. Prediction for (u, i) uses the residuals of the similar users who rated i . Because there are only 610 users, the similarity matrix is tiny and you can even compute it densely.

```
class UserKNN(Recommender):
    name = "user_knn"
    def __init__(self, bias, k=40, shrink=10, damp=1.0):
        self.bias, self.k, self.shrink, self.damp = bias, k, shrink, damp
    def fit(self, train, n_users, n_items):
        self.E, self.B = residual_matrix(train, self.bias)
        self.W = topk_similarity(self.E, self.B, self.k, self.shrink) # users x users
        self.Wabs = abs(self.W)
        return self
    def _num_den(self, u):
        return (self.W[u] @ self.E).toarray().ravel(), (self.Wabs[u] @ self.B).toarray().ravel()
    def score_user(self, u):
        num, den = self._num_den(u)
        return num / (den + self.damp)
    def predict(self, u, i):
        out = np.empty(len(u)); cache = {}
        for n, (uu, ii) in enumerate(zip(u, i)):
            if uu not in cache: cache[uu] = self._num_den(uu)
            num, den = cache[uu]
            out[n] = (self.bias.mu + self.bias.bu[uu] + self.bias.bi[ii]
                     + num[ii] / (den[ii] + self.damp))
        return out
```

6.6 Hyperparameters to tune on validation

Parameter	Range to try	Effect
k (neighbours)	10, 20, 40, 80, 150	Too small: noisy. Too large: dilutes with weak neighbours
shrink	0, 10, 25, 50, 100	Penalises similarities built on tiny overlap
damp	0.1, 0.5, 1, 2, 5	Penalises items supported by little neighbour evidence
Residual vs raw ratings	both	Shows why bias removal matters (include in README)

Alternative scoring for ranking: instead of residuals, use implicit feedback. Let r_u be a binary vector (1 for rated items with rating at least 4). $\text{Score} = W @ r_u$. This often gives better NDCG because ranking is about "will they watch/like", not about calibrated rating deviation. Implement it as a flag `mode="implicit"` and report both.

6.7 Checkpoint and mistakes

- **Done when:** both kNN models have tuned configurations and results in `results_val.csv`. You have a table showing RMSE/NDCG for k and shrink.
- **Expected behaviour:** kNN RMSE should be a bit better than the bias model; its NDCG should beat content-based and approach or beat popularity.
- **Mistake:** computing similarity on the dense 9.7k x 9.7k matrix in one shot (about 750 MB in float64). The chunked function above avoids this.
- **Mistake:** including validation or test ratings in the similarity matrix.
- **Slow predict():** predicting 20k ratings loops over users. That is okay; the cache means each user is computed once.

Phase 7: Matrix Factorization from Scratch

Estimated time: 8 to 10 hours

Goal of this phase: Learn a hidden "taste vector" for every user and a "property vector" for every movie, so that their dot product plus biases predicts the rating. This is the core algorithm behind the Netflix Prize.

7.1 The model

Predicted rating: $\hat{r}(u,i) = \mu + b_u + b_i + p_u \cdot q_i$, where p_u and q_i are vectors of length f (the number of latent factors, for example 32 or 64). Intuitively, one factor might end up meaning "action versus romance" and another "serious versus light", but the model discovers these on its own.

Training minimises the regularised squared error over known ratings: sum of $(r_{ui} - \hat{r}_{ui})^2 + \text{reg} \times (b_u^2 + b_i^2 + |p_u|^2 + |q_i|^2)$. We use stochastic gradient descent (SGD): visit ratings in random order, compute the error $e = r - \hat{r}$, and nudge the parameters:

```
b_u += lr * (e - reg * b_u)
b_i += lr * (e - reg * b_i)
p_u += lr * (e * q_i - reg * p_u)      # using the OLD q_i
q_i += lr * (e * p_u_old - reg * q_i)  # using the OLD p_u
```

7.2 Fast SGD with numba

A pure Python loop over 70k ratings x 64 factors x 30 epochs is too slow. The numba decorator compiles the same loop to machine code and runs it in a second or two per epoch.

```
# src/models/mf.py
import numpy as np
from numba import njit
from .base import Recommender

@njit(cache=True)
def sgd_epoch(u, i, r, order, mu, bu, bi, P, Q, lr, reg):
    f = P.shape[1]
    sq = 0.0
    for t in order:
        a = u[t]; b = i[t]
        pred = mu + bu[a] + bi[b]
        for k in range(f):
            pred += P[a, k] * Q[b, k]
        e = r[t] - pred
        sq += e * e
        bu[a] += lr * (e - reg * bu[a])
        bi[b] += lr * (e - reg * bi[b])
        for k in range(f):
            pa = P[a, k]; qb = Q[b, k]
            P[a, k] += lr * (e * qb - reg * pa)
            Q[b, k] += lr * (e * pa - reg * qb)
    return sq / len(order)

class BiasedMF(Recommender):
    name = "mf"
    def __init__(self, n_factors=64, lr=0.005, reg=0.05, epochs=40, init_std=0.05,
                  seed=42, patience=4, verbose=True):
        self.f, self.lr, self.reg, self.epochs = n_factors, lr, reg, epochs
        self.init_std, self.seed, self.patience, self.verbose = init_std, seed, patience, verbose

    def fit(self, train, n_users, n_items, val=None):
        rng = np.random.default_rng(self.seed)
        u = train.u.values.astype(np.int64); i = train.i.values.astype(np.int64)
        r = train.rating.values.astype(np.float64)
        self.mu = r.mean()
        self.bu = np.zeros(n_users); self.bi = np.zeros(n_items)
        self.P = rng.normal(0, self.init_std, (n_users, self.f))
        self.Q = rng.normal(0, self.init_std, (n_items, self.f))
        self.history, best, bad = [], np.inf, 0
        for ep in range(self.epochs):
```

```

order = rng.permutation(len(r))
train_mse = sgd_epoch(u, i, r, order, self.mu, self.bu, self.bi,
                      self.P, self.Q, self.lr, self.reg)
row = {"epoch": ep + 1, "train_rmse": np.sqrt(train_mse)}
if val is not None:
    pv = np.clip(self.predict(val.u.values, val.i.values), 0.5, 5.0)
    row["val_rmse"] = float(np.sqrt(np.mean((val.rating.values - pv) ** 2)))
    if row["val_rmse"] < best - 1e-4:
        best, bad = row["val_rmse"], 0
        self._best = (self.bu.copy(), self.bi.copy(), self.P.copy(), self.Q.copy())
    else:
        bad += 1
self.history.append(row)
if self.verbose: print(row)
if val is not None and bad >= self.patience:
    self.bu, self.bi, self.P, self.Q = self._best # restore best epoch
    break
return self

def predict(self, u, i):
    return self.mu + self.bu[u] + self.bi[i] + np.einsum("ij,ij->i", self.P[u], self.Q[i])

def score_user(self, u):
    return self.bi + self.Q @ self.P[u] # mu and bu are constant for a given user

```

7.3 Training procedure

1. Fit on **train**, pass **val** for early stopping. Plot train_rmse and val_rmse against epoch (the training-curve plot is a must-have README figure).
2. Healthy curves: both fall, then val flattens or rises while train keeps falling. That gap is overfitting; early stopping picks the best epoch.
3. If both curves explode to NaN, the learning rate is too high. Divide it by 2 or 5.
4. If val RMSE barely moves from the bias baseline, the learning rate is too low or regularisation too high.

7.4 Hyperparameter search

Parameter	Values to try	Notes
n_factors	8, 16, 32, 64, 128	More factors need more regularisation
reg	0.01, 0.03, 0.05, 0.1, 0.2	Most important parameter; tune first
lr	0.002, 0.005, 0.01	Lower is safer, needs more epochs
epochs	up to 60 with early stopping	Let early stopping decide
init_std	0.01, 0.05, 0.1	Small random init breaks symmetry

Do a random search of 20 to 30 combinations, log each trial (params, best val RMSE, val NDCG@10) in a DataFrame, and save it to `reports/mf_search.csv`. Plot val RMSE against reg for different n_factors to show the interaction.

RMSE and ranking disagree. The configuration with the best val RMSE is not always the best val NDCG@10. Select the final MF configuration for the metric you care about in each use. Typically you keep one MF tuned for RMSE (rating prediction) and note that for ranking you pick by NDCG@10. State this choice explicitly.

7.5 Interpreting the latent factors (a great README section)

For the top 3 factors by variance, list the 10 movies with the highest and the 10 with the lowest Q values. Compare the extremes. You will often see something like blockbuster action on one side and art-house drama on the other. Plot movies on the first two factors as a scatter and label a few well-known titles. This turns an abstract model into a story that interviewers remember.

7.6 Optional: compare with a library

Install scikit-surprise or the "implicit" library and compare your numbers with their SVD. If your RMSE is within about 0.01 of the library, your implementation is correct. Note that surprise sometimes has install problems with new numpy versions; this comparison is a sanity check, not a requirement.

7.7 Checkpoint and mistakes

- **Done when:** a tuned MF is saved to artifacts/mf.joblib, the training-curve plot exists, results appended, and the factor-interpretation table is in the notebook.
- **Expected behaviour:** MF RMSE is better than the bias baseline and kNN (a ballpark improvement of 0.02 to 0.06). NDCG@10 is usually the best or near best of single models.
- **Mistake:** updating P and Q using already-modified values within the same step (hence the `pa`, `qb` temporaries in the code).
- **Mistake:** forgetting to clip predictions to [0.5, 5.0] before RMSE.
- **Mistake:** tuning on the test set. Early stopping must use `val`.
- **Cold users or items:** a user or item with no train ratings keeps a near-zero vector, so MF falls back to μ plus biases. That is acceptable and honest; Phase 8 handles it properly.

Phase 8: Hybrid Model and Cold-Start Handling

Estimated time: 6 to 8 hours

Goal of this phase: Blend the individual models so their weaknesses cancel out, and make the system work for users and movies it has never seen.

8.1 Why hybrid

Model	Strength	Weakness
Popularity	Robust, works for everyone	Not personal, low novelty and coverage
Content-based	Works for new items, explainable	Over-specialises (more of the same genre)
Item/User kNN	Simple, explainable ("because you liked X")	Weak on sparse users, no new items
Matrix factorization	Best accuracy, captures subtle taste	Needs history, opaque

A hybrid combines the scores. The simplest effective method is a **weighted blend of normalised scores**.

8.2 Score normalisation

Models output scores on completely different scales (MF gives rating-like values, kNN gives small residuals, content gives cosine values). Before blending, convert each model's scores for one user to z-scores: subtract the user's mean score over items and divide by the standard deviation. This puts every model on the same "how many standard deviations above average" scale.

8.3 Fast weight search with precomputed score matrices

Scoring every user with every model once, caching the matrices, and then trying weight combinations as matrix additions turns a search of hundreds of combinations from hours into seconds.

```
# src/models/hybrid.py
import itertools
import numpy as np
from .base import Recommender

def zscore(s):
    s = np.asarray(s, dtype=np.float64)
    return (s - s.mean()) / (s.std() + 1e-9)

def score_matrix(model, user_idx):
    return np.vstack([zscore(model.score_user(u)) for u in user_idx]).astype(np.float32)

class MatrixScorer(Recommender):
    """Wraps a precomputed (n_users x n_items) score matrix as a model."""
    def __init__(self, Z, name="blend"):
        self.Z = Z
        self.name = name
    def score_user(self, u):
        return self.Z[u]

def weight_grid(n_models, step=0.1):
    steps = int(round(1 / step))
    for combo in itertools.product(range(steps + 1), repeat=n_models):
        if sum(combo) == steps:
            yield tuple(c * step for c in combo)

# --- usage in notebook 07 ---
# models = {"mf": mf, "item_knn": iknn, "content": cb, "pop": pop_count}
# Z = {name: score_matrix(m, range(n_users)) for name, m in models.items()}
# best = (-1, None)
# for w in weight_grid(len(models), 0.1):
#     blend = sum(wi * Z[n] for wi, n in zip(w, models))
#     res = evaluate_ranking(MatrixScorer(blend), train, val, n_items)
#     if res["ndcg@10"] > best[0]: best = (res["ndcg@10"], w)
# print(best)
```

Log every weight combination and its NDCG@10 to a CSV. Plot a heatmap for two of the weights with the others fixed. Also do an **ablation**: remove one model at a time (set its weight to zero, redistribute) and report how much NDCG@10 drops. That table shows which components earn their place.

8.4 Rating-prediction hybrid

For RMSE, blend the predicted ratings of bias, item-kNN, user-kNN, and MF with a ridge regression fitted on the **validation** predictions (features = each model's prediction, target = true rating). This is called stacking. Use cross-validation on the validation set to check the blend generalises, then fit the final coefficients on all of validation.

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_predict

X = np.column_stack([m.predict(val.u.values, val.i.values) for m in rating_models])
y = val.rating.values
stack = Ridge(alpha=1.0, positive=True) # positive weights keep it interpretable
cv_pred = cross_val_predict(stack, X, y, cv=5)
print("stack CV RMSE:", rmse(y, np.clip(cv_pred, 0.5, 5)))
stack.fit(X, y)
print(dict(zip([m.name for m in rating_models], stack.coef_.round(3)), stack.intercept_))
```

8.5 The cold-start problem

Cold start has three forms and a good project handles all three:

Case	Problem	Strategy in this project
New user, zero ratings	No signal at all	Show popular movies; let the user pick favourite genres; show a rating onboarding screen
New user, few ratings (3 to 20)	Not enough data to train a user vector	Fold-in : compute their vector from the items they rated, without retraining
New item	No ratings so CF cannot score it	Content-based similarity using genres and tags; optional exploration boost

8.6 Fold-in for a new user

```
# new_user: arrays `items` (item indices) and `ratings` (floats) from onboarding

def fold_in_mf(mf, bias_lambda, items, ratings, lam=1.0):
    res = ratings - mf.mu - mf.bi[items]
    bu = res.sum() / (bias_lambda + len(res)) # user bias
    Qs = mf.Q[items] # (n_rated x f)
    A = Qs.T @ Qs + lam * np.eye(Qs.shape[1])
    p = np.linalg.solve(A, Qs.T @ (res - bu)) # ridge regression for p_u
    return mf.bi + mf.Q @ p # scores for all items

def fold_in_knn(knn, bias, n_items, items, ratings):
    res = ratings - bias.mu - bias.bi[items]
    bu = res.sum() / (15 + len(res))
    e = np.zeros(n_items); r = np.zeros(n_items)
    e[items] = res - bu; r[items] = 1
    return (knn.W @ e) / (knn.Wabs @ r + knn.damp)

def fold_in_content(cb, items, ratings):
    w = ratings - ratings.mean()
    if np.allclose(w, 0): w = np.ones(len(items))
    prof = np.asarray(cb.X[items].T @ w).ravel()
    return np.asarray(cb.X @ prof).ravel()
```

Why fold-in works: for a fixed set of item vectors Q , finding the best user vector is just a small ridge regression problem with a closed-form answer. No gradient descent and no retraining is needed, so the web app can respond in milliseconds.

8.7 The engine class used by the web app

```
class HybridEngine:
    def __init__(self, mf, iknn, cb, pop, bias, weights, n_items):
        self.mf, self.iknn, self.cb, self.pop, self.bias = mf, iknn, cb, pop, bias
        self.w, self.n_items = weights, n_items          # dict like {"mf":0.5,"iknn":0.3,...}

    def _blend(self, parts):
        return sum(self.w[k] * zscore(v) for k, v in parts.items() if self.w.get(k, 0) > 0)

    def score_user(self, u):
        # existing user
        return self._blend({"mf": self.mf.score_user(u), "iknn": self.iknn.score_user(u),
                           "content": self.cb.score_user(u), "pop": self.pop.score_user(u)})

    def score_new_user(self, items, ratings):
        # cold-start user with a few ratings
        items, ratings = np.asarray(items), np.asarray(ratings, float)
        if len(items) == 0:
            return zscore(self.pop.score_user(0))
        return self._blend({"mf": fold_in_mf(self.mf, 15, items, ratings),
                           "iknn": fold_in_knn(self.iknn, self.bias, self.n_items, items, ratings),
                           "content": fold_in_content(self.cb, items, ratings),
                           "pop": self.pop.score_user(0)})

    def similar_items(self, i, n=10):
        # item-to-item recommendations
        s = zscore(self.iknn.W[i].toarray().ravel()) * 0.6 + zscore(self.mf.Q @ self.mf.Q[i]) * 0.4
        s[i] = -np.inf
        top = np.argpartition(-s, n)[:n]
        return top[np.argsort(-s[top])]
```

Place the three `fold_in` functions and the `HybridEngine` class in `src/models/hybrid.py` next to the blending helpers, so that `joblib` can find the class again when the web app loads the saved engine.

For a brand-new user the weights found in 8.3 (tuned for users with long history) may not be best. Run the cold-start experiment in 8.8 and, if needed, store a second weight set for users with fewer than about 10 ratings (typically more popularity and content weight).

8.8 Cold-start experiment (strong portfolio material)

Simulate new users using existing users. For n in 0, 1, 3, 5, 10, 20, 50: for every user keep only their first n train ratings (chronologically), fold in, score, and evaluate NDCG@10 against that user's test items. Do this for popularity, content, kNN, MF, and the hybrid. Plot NDCG@10 against n with one line per model.

- Expected picture: popularity is flat; content and kNN rise quickly; MF needs more ratings but ends highest; the hybrid should be near the top at every n .
- The crossover points (for example "MF overtakes content after about 10 ratings") become a concrete design statement: "the app asks new users for at least 5 ratings".

8.9 Checkpoint and mistakes

- **Done when:** hybrid weights chosen on validation; ablation table and cold-start plot saved; `HybridEngine` passes a smoke test that scores both an existing user and a fake new user.
- **Mistake:** picking the hybrid weights on the test set.
- **Mistake:** z-scoring with `np.std` of an all-zero score vector (kNN for a user with no overlapping neighbours). The $1e-9$ term prevents the divide-by-zero.
- **Mistake:** letting items already rated by the new user appear in their recommendations. Always mask them.

Phase 9: Final Experiments, Error Analysis and the Test Evaluation

Estimated time: 6 to 8 hours

Goal of this phase: Produce trustworthy final numbers, understand where the system fails, and write the findings.

9.1 Final protocol

1. Freeze every hyperparameter and every blend weight from the validation phase. No more changes after this point.
2. Refit each model on **train + val** combined. For MF, use a fixed number of epochs equal to the best epoch found earlier (there is no separate validation set for early stopping now). Re-fit the bias model as well, because kNN residuals depend on it.
3. Evaluate once on the test set. When computing ranking metrics, the seen set is train + val.
4. Save the numbers to `reports/results_test.csv` using the same schema as validation.

9.2 Final results table (template)

Model	RMSE	MAE	P@10	R@10	NDCG@10	Coverage	Novelty
Random							
Popularity							
Bias							
Content-based	n/a	n/a					
Item-kNN							
User-kNN							
Matrix factorization							
Hybrid							

Fill this with your real numbers. Bold the best value in each column in the README.

9.3 Statistical confidence

A model that wins by 0.002 NDCG may just be lucky. Add two checks:

- **Seed variance:** train MF with 5 different seeds; report mean and standard deviation of RMSE and NDCG@10.
- **Bootstrap over users:** modify `evaluate_ranking` to also return the per-user NDCG list. Resample users with replacement 1,000 times, compute the mean each time, and report the 2.5th and 97.5th percentiles as a 95 percent confidence interval. If the intervals of two models overlap heavily, say the difference is not clear.

```
def bootstrap_ci(per_user, n_boot=1000, seed=0):
    rng = np.random.default_rng(seed)
    per_user = np.asarray(per_user)
    means = [rng.choice(per_user, len(per_user), replace=True).mean() for _ in range(n_boot)]
    return np.percentile(means, [2.5, 97.5])
```

9.4 Error analysis - find where the model is weak

1. **By user activity:** bucket users by number of train ratings (20-50, 50-100, 100-300, 300+). Plot RMSE and NDCG@10 per bucket per model. Expect light users to be harder.

2. **By item popularity:** bucket test ratings by the item's train rating count (0, 1-5, 6-20, 21-100, 100+). Plot RMSE. Expect rare items to have the largest errors.
3. **By genre:** RMSE per genre. Are some genres systematically harder?
4. **By rating value:** mean prediction for each true rating 0.5, 1.0, ..., 5.0. Models usually pull extremes toward the centre; show the calibration plot.
5. **Popularity bias:** plot the distribution of recommended items by popularity decile for each model, versus the distribution in the test set. Report the share of recommendations coming from the top 10 percent of movies.
6. **Diversity:** average pairwise content distance (1 minus cosine from your content vectors) among each user's top 10. Low diversity means a filter bubble.
7. **Worst predictions:** list the 20 test ratings with the largest absolute error. Read them. Are they user quirks, weird movies, or data issues?

9.5 Qualitative evaluation (yourself as a user)

Numbers do not tell you if recommendations feel good. Create a pseudo-user from 15 to 20 movies you really know and have opinions about, rate them, and use the fold-in path to get the top 10. Judge each list for relevance, variety, and obvious mistakes. Screenshot the best and worst cases into the README. If you watch cricket movies, sports dramas, or anything niche, check how the system treats them; these examples make an honest case study.

9.6 Write the findings (the part reviewers read)

Write 1 to 2 pages in `reports/findings.md` answering:

- Which model won on rating prediction, and which on ranking? Are they the same? Why not?
- How much did the hybrid add over the best single model? Is the gain outside the confidence interval?
- Where does the system fail (cold items, light users, niche genres)? What did you do about it?
- What did the popularity baseline teach you about evaluating recommenders?
- What would you do next with more time or data?

9.7 Checkpoint and mistakes

- **Done when:** `results_test.csv`, all error-analysis plots in `reports/plots/`, and `findings.md` exist.
- **Mistake:** reporting validation numbers as final. Always report test.
- **Mistake:** hiding negative results. If the hybrid does not beat MF, report it honestly and discuss why; that is still a valid project.

Phase 10: Flask Web Application

Estimated time: 8 to 10 hours

Goal of this phase: Turn the trained system into a usable product: search a movie, see similar movies, get recommendations for a known user, or rate a few movies and get personal recommendations immediately.

10.1 Production training script

Write `src/build_artifacts.py` that fits all models on **all** available ratings (train + val + test) with the chosen hyperparameters and writes the artifacts used by the app. The test set was only for evaluation; for the deployed product you want to use every rating.

```
# src/build_artifacts.py (run: python -m src.build_artifacts)
import joblib, numpy as np, pandas as pd
from .config import DATA_PROC, ARTIFACTS
from .models.baselines import BiasModel, Popularity
from .models.content import build_item_features, ContentBased
from .models.knn import ItemKNN
from .models.mf import BiasedMF
from .models.hybrid import HybridEngine

movies = pd.read_parquet(DATA_PROC / "movies.parquet")
full = pd.concat([pd.read_parquet(DATA_PROC / f"{s}.parquet") for s in ("train", "val", "test")])
n_users, n_items = full.u.max() + 1, len(movies)

bias = BiasModel().fit(full, n_users, n_items)
pop = Popularity("count").fit(full, n_users, n_items)
X, _ = build_item_features(movies)
cb = ContentBased(X).fit(full, n_users, n_items)
iknn = ItemKNN(bias, k=BEST_K, shrink=BEST_SHRINK).fit(full, n_users, n_items)
mf = BiasedMF(n_factors=BEST_F, reg=BEST_REG, lr=BEST_LR, epochs=BEST_EPOCHS, verbose=False)
mf.fit(full, n_users, n_items)

engine = HybridEngine(mf, iknn, cb, pop, bias, weights=BEST_WEIGHTS, n_items=n_items)
joblib.dump(engine, ARTIFACTS / "engine.joblib", compress=3)

seen = full.groupby("u").i.apply(np.array).to_dict()
joblib.dump(seen, ARTIFACTS / "seen_by_user.joblib")
```

Replace the `BEST_` constants with the numbers from your validation experiments (or read them from a small JSON file saved in Phase 9). Check the size of `engine.joblib`; the item-kNN sparse matrix and MF matrices are normally well under 50 MB.

10.2 API design

Endpoint	Method	Purpose	Response
/	GET	Serve the web page	HTML
/api/search?q=toy	GET	Title autocomplete	List of {movie_id, title, genres}
/api/similar/<movie_id>?n=10	GET	Movies similar to one movie	List of movie cards with scores
/api/recommend/user/<user_id>?n=10	GET	Top-N for an existing MovieLens user	List of movie cards
/api/recommend/new	POST	Cold-start: body has a list of {movie_id, rating}	List of movie cards
/api/health	GET	Monitoring and deployment check	{status: ok}

10.3 app.py

```
# app/app.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parents[1]))
import numpy as np, pandas as pd, joblib
from flask import Flask, jsonify, request, render_template
from src.config import ARTIFACTS, DATA_PROC

app = Flask(__name__)
movies = pd.read_parquet(DATA_PROC / "movies.parquet")
movies["title_lower"] = movies.title.str.lower()
maps = joblib.load(ARTIFACTS / "id_maps.joblib")
engine = joblib.load(ARTIFACTS / "engine.joblib")
seen_by_user = joblib.load(ARTIFACTS / "seen_by_user.joblib")
movie2idx = maps["item2idx"]

def card(i, score=None):
    r = movies.iloc[int(i)]
    d = {"movie_id": int(r.movieId), "title": r.title, "genres": r.genres.replace("|", ", ")}
    if score is not None: d["score"] = round(float(score), 3)
    return d

def top_n(scores, exclude, n):
    s = np.array(scores, dtype=float)
    if len(exclude): s[np.asarray(exclude, dtype=int)] = -np.inf
    n = min(n, len(s) - 1)
    idx = np.argpartition(-s, n)[:n]
    idx = idx[np.argsort(-s[idx])]
    return [card(i, s[i]) for i in idx]

def get_n(default=10, cap=30):
    try: return max(1, min(int(request.args.get("n", default)), cap))
    except ValueError: return default

@app.route("/")
def index(): return render_template("index.html")

@app.get("/api/health")
def health(): return jsonify(status="ok")

@app.get("/api/search")
def search():
    q = request.args.get("q", "").strip().lower()
    if len(q) < 2: return jsonify([])
    hit = movies[movies.title_lower.str.contains(q, regex=False)].head(15)
    return jsonify([card(i) for i in hit.index])

@app.get("/api/similar/<int:movie_id>")
def similar(movie_id):
    if movie_id not in movie2idx: return jsonify(error="unknown movie"), 404
    i = movie2idx[movie_id]
    idx = engine.similar_items(i, get_n())
    return jsonify([card(j) for j in idx])

@app.get("/api/recommend/user/<int:user_id>")
def rec_user(user_id):
    u = maps["user2idx"].get(user_id)
    if u is None: return jsonify(error="unknown user (valid ids are 1 to 610)"), 404
    return jsonify(top_n(engine.score_user(u), seen_by_user.get(u, []), get_n()))

@app.post("/api/recommend/new")
def rec_new():
    body = request.get_json(silent=True) or {}
    rated = body.get("ratings", [])
    items, ratings = [], []
    for x in rated[:100]:
        try:
            mid, rt = int(x["movie_id"]), float(x["rating"])
        except (KeyError, TypeError, ValueError):
            return jsonify(error="bad rating entry"), 400
        if mid in movie2idx and 0.5 <= rt <= 5.0:
            items.append(movie2idx[mid]); ratings.append(rt)
    if len(items) < 3:
```

```

        return jsonify(error="please rate at least 3 known movies"), 400
    scores = engine.score_new_user(items, ratings)
    return jsonify(top_n(scores, items, get_n()))

if __name__ == "__main__":
    app.run(debug=True)

```

10.4 Frontend

Keep it a single page with three tabs. You already know Bootstrap and fetch/jQuery from your earlier Flask project, so reuse that style. Required UI pieces:

1. **Search box with autocomplete:** on every key press (debounced by 250 ms) call `/api/search` and show a dropdown of titles.
2. **"Similar movies" tab:** choose a movie, call `/api/similar`, show cards (title, genres, score).
3. **"For a MovieLens user" tab:** number input 1 to 610 and a Recommend button.
4. **"Rate and discover" tab (cold start):** the user searches and adds at least 5 movies, gives each a star rating (1 to 5), then presses Get Recommendations; the page POSTs to `/api/recommend/new`.
5. Loading spinner, empty-state message, and readable error messages (show the server's "error" text).
6. Optional: poster images. `links.csv` gives `tmdbId`; register for a free TMDb API key and call their image endpoint from the **server** (never expose your key in JavaScript). Cache results in a dict or a small SQLite table.

```

// static/js/main.js - cold-start submit
async function getRecs(ratings) {
  const res = await fetch("/api/recommend/new?n=12", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({ratings}) // [{movie_id: 1, rating: 5}, ...]
  });
  const data = await res.json();
  if (!res.ok) { showError(data.error); return; }
  renderCards(data);
}

```

10.5 Run and test

- Run `python app/app.py` and open `http://127.0.0.1:5000`.
- Test every endpoint with the browser and with curl (for example `curl http://127.0.0.1:5000/api/recommend/user/1`).
- Add 3 small automated tests with Flask's test client: health returns 200; unknown user returns 404; posting 2 ratings returns 400.
- Check latency: each request should take under 200 ms. If it is slow, profile `score_user`; the usual culprit is converting large sparse matrices to dense inside the request.

10.6 Checkpoint and mistakes

- **Done when:** all four tabs work end to end and a friend can use the cold-start flow without instructions.
- **Mistake:** training or loading models inside a request handler. Load once at startup.
- **Mistake:** pickle compatibility. joblib files created with one scikit-learn / numpy version may fail on another. Use the same versions (`requirements.txt`) everywhere.
- **Mistake:** leaving `debug=True` in production. Use gunicorn in deployment (Phase 12).

Phase 11: Power BI Dashboard

Estimated time: 4 to 5 hours

Goal of this phase: Present the data exploration and the model comparison as an interactive dashboard, using the Power BI skill you already have.

11.1 Export tidy CSVs from Python

File	Contents	Used for
bi_ratings_summary.csv	One row per rating bucket / year / genre with counts and means	EDA page
bi_movies.csv	movieId, title, year, genres (one row per movie-genre), rating_count, mean_rating	Movie explorer
bi_users.csv	userId, n_ratings, mean_rating, activity_bucket	User segmentation
results_test.csv	model, split, metric, value, params	Model comparison page
bi_error_by_bucket.csv	model, bucket_type, bucket, rmse, ndcg	Error analysis page
bi_cold_start.csv	model, n_ratings, ndcg@10	Cold-start page
bi_recs_sample.csv	userId, rank, movieId, title, model	Recommendation inspector

Make every file "long and tidy": one observation per row and one column per attribute. Power BI is much easier to use with tidy tables.

11.2 Dashboard pages

Page	Visuals	Slicers
1. Data Overview	KPI cards (users, movies, ratings, sparsity), rating distribution column chart, ratings per year line chart	Year, genre
2. Long Tail	Line chart of sorted movie rating counts, share-of-ratings by popularity decile, scatter of count vs mean rating	Genre
3. Model Comparison	Clustered bar chart for RMSE and NDCG@10 by model, matrix table of all metrics with conditional formatting, coverage vs NDCG scatter	Metric, split
4. Error Analysis	RMSE by user-activity bucket and by item-popularity bucket for each model (line chart), genre heat map	Model
5. Cold Start	Line chart: NDCG@10 vs number of known ratings, one line per model	Model
6. Recommendation Inspector	Table showing top-10 recommended titles for a selected user and model	User, model

11.3 Useful DAX measures

```
Avg NDCG@10 =  
CALCULATE ( AVERAGE ( Results[value] ), Results[metric] = "ndcg@10" )  
  
Best Model NDCG =  
VAR t = FILTER ( Results, Results[metric] = "ndcg@10" && Results[split] = "test" )  
RETURN MAXX ( TOPN ( 1, t, Results[value], DESC ), Results[model] )  
  
Sparsity % =  
1 - DIVIDE ( COUNTROWS ( Ratings ),  
    DISTINCTCOUNT ( Ratings[userId] ) * DISTINCTCOUNT ( Movies[movieId] ) )  
  
Lift vs Popularity =  
VAR pop = CALCULATE ( AVERAGE ( Results[value] ),  
    Results[model] = "popularity_count", Results[metric] = "ndcg@10" )  
RETURN DIVIDE ( [Avg NDCG@10] - pop, pop )
```

The Best Model measure is a starting point; adjust the model-name filter to match your own naming. Always sanity check measures with a card visual that shows the number next to your Python result.

11.4 Design rules

- One accent colour for your best model, grey for the rest, so the reader sees the winner instantly.
- Sort bars descending, label values directly, and avoid 3D charts or more than 6 colours.
- Add a title that states the finding (for example "Hybrid improves NDCG@10 over popularity") rather than just naming the chart.
- Export screenshots of pages 1, 3 and 5 into reports/dashboard/ for the README, and commit the .pbix file only if it is under 25 MB.

11.5 Checkpoint

- Six-page .pbix opens with the data refreshing from the CSV folder, and the numbers in the Model Comparison page exactly match results_test.csv.

Phase 12: Documentation, Testing, Deployment and Resume

Estimated time: 6 to 8 hours

Goal of this phase: Turn working code into a portfolio piece that a recruiter can understand in two minutes and a hiring manager can trust in ten.

12.1 README structure

1. **Title and one-line pitch:** "CineMatch: hybrid movie recommender (content + collaborative + matrix factorization) with live demo".
2. **Demo:** link to the deployed app, plus a 20 to 30 second GIF of the cold-start flow.
3. **Results at a glance:** the final results table and the most important chart (cold-start curve or model comparison).
4. **Problem and dataset:** what is being recommended, dataset size, sparsity, key EDA insight.
5. **Methodology:** split protocol (per-user temporal), models, metrics, with short explanations.
6. **Key findings:** 4 to 6 bullets from findings.md.
7. **How to run:** setup, data download, `python -m src.build_artifacts`, `python app/app.py`.
8. **Repository map and limitations:** what the data cannot tell you (explicit ratings only, one dataset, no business metrics such as clicks or watch time).
9. **Future work:** implicit feedback, neural models, bigger data, A/B test.

12.2 Code quality checklist

- Docstrings on every public function; type hints where they help.
- No notebook cell depends on hidden state; "Restart and run all" works.
- pytest passes for metrics, split leakage test, and API endpoints.
- A seed is set everywhere randomness is used, and results are reproducible to the third decimal.
- requirements.txt pins versions (for example `pip freeze` output trimmed to the libraries you import).
- No data files over 100 MB in git. The raw dataset is downloaded by a short script (`scripts/download_data.py`) rather than committed.

```
# tests/test_split.py - leakage test
def test_no_future_leakage(train, test):
    last_train = train.groupby("userId").timestamp.max()
    first_test = test.groupby("userId").timestamp.min()
    assert (last_train.reindex(first_test.index) <= first_test).all()

def test_no_overlap(train, val, test):
    key = lambda d: set(zip(d.u, d.i))
    assert not (key(train) & key(val))
    assert not (key(train) & key(test))
    assert not (key(val) & key(test))
```

12.3 Deployment

Two free and easy options. Both host Flask apps from a GitHub repository.

Option	Steps	Notes
Render.com (Web Service)	Connect repo; build command <code>pip install -r requirements.txt</code> ; start command <code>gunicorn app:app</code>	Free tier sleeps after inactivity, so the first request can take about a minute

Option	Steps	Notes
Hugging Face Spaces (Docker or Gradio/Flask)	Create a Space, add a Dockerfile that installs requirements and runs gunicorn on port 7860	Stays up reasonably well; good visibility for ML portfolios

```
# Procfile (Render / Heroku-style)
web: gunicorn app.app:app --workers 2 --timeout 60

# Dockerfile (Hugging Face Spaces)
FROM python:3.11-slim
WORKDIR /code
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 7860
CMD ["gunicorn", "app.app:app", "--bind", "0.0.0.0:7860", "--workers", "2", "--timeout", "60"]
```

- The deployed machine has no GPU and only a few hundred MB of RAM. The artifacts must load in memory comfortably; check with `engine.joblib` size.
- Do not run training on the server. Commit the prebuilt artifacts (or download them on start from a release asset).
- Hide secrets (TMDB key) in environment variables, not in code.
- Test the live URL from your phone and add it to the README.

12.4 Resume bullets (fill the numbers with your actual results)

```
CineMatch - Hybrid Movie Recommender (Python, scikit-learn, NumPy, Flask, Power BI)
- Built an end-to-end recommender on MovieLens (100k ratings) combining TF-IDF content filtering,
  item-based collaborative filtering and a from-scratch biased matrix factorization (numba SGD).
- Designed a leakage-free per-user temporal evaluation; hybrid improved NDCG@10 by X% over a
  popularity baseline and reduced RMSE from A to B.
- Implemented cold-start fold-in for new users and items; analysed accuracy vs. number of known
  ratings and popularity bias/coverage across models.
- Deployed a Flask web app with live search, similar-movie and personalised recommendation
  endpoints; built a 6-page Power BI dashboard for model comparison and error analysis.
```

12.5 Interview questions you should now be able to answer

Full list in Appendix C. Practise saying the answers out loud for 60 seconds each.

12.6 Final checkpoint

- Live URL works. README renders with images. Repo has a license (MIT is fine).
- Pinned post on GitHub profile, LinkedIn post with a screenshot and 3 learning points, and the project listed under Projects on your resume.

Phase 13: Optional Extensions (Pick One or Two)

Estimated time: Each 8 to 15 hours

Goal of this phase: Go beyond the base project to show depth. Do these only after Phases 1 to 12 are complete and published.

Extension	What to build	What you learn
A. Implicit feedback (BPR / ALS)	Treat rating of 4 or more as a positive interaction; train with the "implicit" library (ALS) or your own BPR in numba; compare ranking metrics	Most real data has clicks, not ratings
B. Neural collaborative filtering	Small PyTorch model with user/item embeddings and an MLP; compare against MF honestly	Why deep models do not always beat MF on small data
C. Time-aware recommendation	Add item bias drift or recency weighting; use last-N-movies as session context	Temporal dynamics (also connects to your time-series learning)
D. Scale-up to MovieLens 1M	Run the same pipeline on ml-1m (about 1M ratings); measure training time and memory	Engineering for scale
E. Explainability	For each recommendation, show "Because you liked X, Y, Z" using the largest contributing neighbours	Trust and debugging
F. Diversity re-ranking (MMR)	Re-rank top 50 to balance relevance and diversity; plot the accuracy-diversity trade-off	Beyond-accuracy goals
G. Offline A/B simulation	Simulate two policies and compare using inverse propensity scoring on logged data	Counterfactual evaluation basics

13.1 Cricket variant (portfolio personalisation)

Since cricket is one of your favourite areas, a strong second recommender project uses the same code with new data: "recommend similar IPL players" (content-based on career stats such as batting average, strike rate, bowling economy, role, and team; find nearest neighbours with cosine similarity on standardised features) or "recommend matches to watch" (users rate matches; item features are teams, venue, and result margin). Public ball-by-ball IPL datasets are available on Kaggle. This shows that you can transfer the pipeline to a new domain, which is more impressive than a second MovieLens project.

Keep the main CineMatch repository focused. Put any extension in a separate folder or repository and link it from the README.

Appendix A: Formula and Concept Cheat Sheet

Concept	Formula / definition
Global mean	μ = average of all training ratings
Bias prediction	$\hat{r}(u,i) = \mu + b_u + b_i$
Regularised item bias	$b_i = \sum_u (r_{ui} - \mu) / (\lambda_i + n_i)$
Regularised user bias	$b_u = \sum_i (r_{ui} - \mu - b_i) / (\lambda_u + n_u)$
Residual	$e_{ui} = r_{ui} - (\mu + b_u + b_i)$
Cosine similarity	$\cos(a,b) = (a \cdot b) / (a \times b)$
Shrunk similarity	$s_{shrunk} = s \times n_{common} / (n_{common} + \lambda)$
kNN residual score	$score(u,i) = \sum_j s_{ij} e_{uj} / (\sum_j s_{ij} + \text{damp})$, j in top-k neighbours of i rated by u
MF prediction	$\hat{r}(u,i) = \mu + b_u + b_i + p_u \cdot q_i$
MF loss	$\sum_{\text{known } (u,i)} (r_{ui} - \hat{r}_{ui})^2 + \text{reg} \times (b_u^2 + b_i^2 + p_u ^2 + q_i ^2)$
SGD update	$\theta = \theta + \text{lr} \times (e \times \text{gradient_term} - \text{reg} \times \theta)$
Fold-in	$p_{new} = (Q_S^T Q_S + \lambda I)^{-1} Q_S^T (r_S - \mu - b_S - b_{new})$
RMSE	$\sqrt{\text{mean}((r - \hat{r})^2)}$
Precision@K	(number of relevant items in top K) / K
Recall@K	(number of relevant items in top K) / (number of relevant items)
DCG@K	$\sum_{\text{rank } r=1..K} \text{rel}_r / \log_2(r+1)$
NDCG@K	DCG@K / ideal DCG@K (between 0 and 1)
Coverage	number of distinct items recommended / catalogue size
Novelty	mean of $-\log_2(p_i)$, where p_i is item i's share of ratings
Sparsity	$1 - (\text{number of ratings}) / (\text{users} \times \text{items})$

Appendix B: Common Pitfalls and How to Avoid Them

Pitfall	Symptom	Fix
Random train/test split	Unrealistically good scores	Per-user temporal split
Tuning on test data	Great test numbers that vanish in production	Use validation only; touch test once
Not masking seen items	Model recommends movies the user already rated	Set seen scores to -inf in the evaluator
Ignoring popularity baseline	Models look good only because no strong baseline exists	Always report popularity (count) in ranking tables
Dense similarity matrix	Out of memory	Chunked sparse top-k similarity
Raw-rating cosine	Everything is similar to everything	Use residuals after removing biases
Unclipped predictions	RMSE inflated, ratings above 5	Clip to 0.5 to 5.0
Index confusion (raw IDs vs matrix positions)	Wrong movies in output, shape errors	One mapping in <code>id_maps.joblib</code> ; always use u, i columns in models
Learning rate too high	Loss becomes NaN	Lower lr by a factor of 2 to 10
Reporting only RMSE	Missing the point of recommendation	Report ranking and beyond-accuracy metrics too

Pitfall	Symptom	Fix
Training in the web request	Slow, crashes under load	Load prebuilt artifacts at startup
Version mismatch for pickles	App fails to load model on server	Pin library versions identical in dev and deploy

Appendix C: Interview Questions with Short Answers

Q: What is the difference between content-based and collaborative filtering?

A: Content-based uses item attributes and a user's own history. Collaborative filtering uses patterns across many users' behaviour and does not need item attributes, but it cannot score items with no interactions.

Q: Why did you use a temporal split instead of a random split?

A: A random split leaks future information. In production we only know the past, so training on earlier ratings and testing on later ones is the honest simulation.

Q: Why is popularity such a strong baseline?

A: Users concentrate on a few popular items (long tail), so recommending those gets many hits. Any personalised model must beat it to justify its complexity.

Q: Why do you compute similarity on residuals?

A: Raw ratings are dominated by user and item bias. Residuals measure deviations from expectation, so similarity reflects shared taste.

Q: Explain matrix factorization in two sentences.

A: It represents each user and item as a short vector so that their dot product (plus biases) approximates the rating. The vectors are learned by minimising regularised squared error on known ratings.

Q: How do you prevent overfitting in MF?

A: L2 regularisation on factors and biases, early stopping on a validation set, and choosing a modest number of factors.

Q: How does your system handle a new user?

A: It uses fold-in: with a handful of ratings, solve a ridge regression for the user vector given fixed item vectors. With zero ratings it falls back to popularity.

Q: RMSE improved but NDCG did not. Why?

A: RMSE measures calibration of predicted ratings across all items; NDCG measures the order at the top. A model can be well calibrated overall and still rank the best items poorly.

Q: What is a filter bubble and how did you measure it?

A: Recommendations that become increasingly similar. I measured coverage, novelty, and intra-list diversity, and optionally tested MMR re-ranking.

Q: What would change with implicit feedback?

A: No negative signals exist, so I would use losses like BPR or confidence-weighted ALS and evaluate purely with ranking metrics.

Q: How would you scale this to 100 million ratings?

A: Use distributed ALS or approximate nearest neighbour search for retrieval, a two-stage retrieve-then-rank architecture, and offline batch plus online serving.

Q: What are the limitations of your evaluation?

A: Offline metrics only, explicit ratings from one platform, no exposure bias correction, small user base. Online A/B testing would be the true test.

Appendix D: Master Checklist

Phase	Done when	Tick
1 Setup	Repo, venv, config.py, first commit	☐
2 EDA	8+ plots, 6+ written findings	☐
3 Data prep	Splits saved, leakage tests pass, id maps saved	☐
4 Evaluation and baselines	metrics.py tested, 4 baseline results logged	☐
5 Content-based	Tuned on val, neighbour lists look sensible	☐
6 kNN CF	Item-kNN and User-kNN tuned, tables for k and shrink	☐
7 Matrix factorization	Training curves, tuned params, factor interpretation	☐
8 Hybrid and cold start	Weights tuned, ablation, cold-start curve, HybridEngine works	☐
9 Final evaluation	results_test.csv, error analysis plots, findings.md	☐
10 Flask app	Four tabs working, API tests pass	☐
11 Power BI	Six pages, numbers match Python	☐
12 Docs and deploy	Live URL, README with GIF, resume bullets, LinkedIn post	☐
13 Extension (optional)	One extension finished in its own folder	☐

If you get stuck: reduce scope, never quality of evaluation. You can drop user-kNN, the Power BI cold-start page, or the optional poster images. You should not drop the temporal split, the baselines, or the test-set discipline; those are what make the project credible.

Glossary

Term	Meaning
Explicit feedback	Direct opinions such as star ratings
Implicit feedback	Behaviour such as clicks, views, purchases
Latent factor	A hidden dimension learned by the model that summarises taste or item properties
Cold start	Situations with no or little interaction data for a user or item
Long tail	The large number of items that each receive few interactions
Leakage	Using information in training that would not be available at prediction time
Regularisation	Penalty that keeps parameters small to reduce overfitting
Fold-in	Computing a new user's vector from their ratings with the item vectors fixed
Shrinkage	Pulling uncertain estimates (small-overlap similarities) toward zero
Exposure bias	Users only rate what they were shown, so missing ratings are not random